

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Варіант №10

Виконала: студентка групи AI-244
Куриленко І.О.

Перевірив: ас. Баранов О.М.

Одеса – 2026

Мета роботи: розгорнути середовище для роботи з локальними та хмарними мовними моделями, дослідити особливості міркувальних та інструктивних моделей, порівняти їхню швидкість і якість, а також експериментально дослідити вплив параметрів `temperature` і `top_p` на стабільність відповідей.

Конфігурація середовища: робота виконувалась у Windows 10. Для локального запуску моделей використовувався Ollama 0.34.1, Python 3.12.10 та віртуальне середовище `venv`. Досліджувалися локальні моделі `llama3.2:3b` і `qwen3:4b`, а як хмарна модель використовувалась `gemini-3.6-flash`.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Для порівняння міркувальної та інструктивної моделей було виконано запити до `qwen3:4b` і `llama3.2:3b` через ендпойнт `/api/generate`. Спочатку моделям було поставлено просте фактичне питання, після чого кожна модель п'ять разів виконала задачу на міркування. Перед серією вимірювань було виконано прогрівальний запит.

Результати виконання простого фактичного запиту наведено в таблиці 1.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	396	5
Час, с	81,72	0,90
Токенів/с	4,91	7,63

Як видно з таблиці 1, обидві моделі правильно визначили столицю України, однак `qwen3:4b` витратила значно більше вихідних токенів і часу на формування відповіді.

Результати задачі на міркування за п'ятьма запусками наведено в таблиці 2.

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	1 / 5
eval_count	1230,4	151,0
Час, с	298,16	23,93

У п'яти запусках llama3.2:3b правильно розв'язала задачу лише один раз. Серед отриманих відповідей були 50 грн, 100 грн, 100 грн, 5 грн і 105 грн. Отже, модель показала нестабільність при виконанні багатокрокового арифметичного міркування. Натомість qwen3:4b правильно відповіла у всіх п'яти запусках, хоча витратила значно більше часу і токенів.

Відповіді на питання (крок 6):

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: у qwen3:4b значна частина токенів витрачається на внутрішні міркування, які зберігаються у полі thinking, хоча користувач бачить лише кінцеву відповідь. Тому навіть для простого фактичного питання модель може згенерувати набагато більше токенів, ніж llama3.2:3b. Це відповідає опису міркувальної моделі в методичці.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: міркувальні моделі корисні на задачах, де потрібно виконувати кілька логічних або математичних кроків, аналізувати умови та перевіряти результат. Для простих фактичних запитів їх використання може бути не виправданим, оскільки вони витрачають більше часу і токенів.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: інструктивну модель навчають насамперед добре виконувати команди користувача і швидко формувати кінцеву відповідь. Міркувальну модель

додатково орієнтують на побудову проміжного процесу розв'язання, що допомагає у складніших задачах.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: ні, сама по собі більша кількість токенів не означає вищий інтелект або якість моделі. Вона показує лише те, що модель виконала більше генерації; результат треба оцінювати також за правильністю, швидкістю та відповідністю завданню.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: швидкість залежить від архітектури й розміру моделі, способу генерації та характеристик комп'ютера. У нашому експерименті qwen3:4b додатково генерувала внутрішні міркування, через що загальний час її відповіді був значно більшим.

Висновок за завданням 1

Міркувальні моделі доцільно використовувати для задач, які потребують логічних обчислень і декількох кроків розв'язання. У моєму експерименті qwen3:4b правильно розв'язала задачу 5 разів із 5, тоді як llama3.2:3b – лише 1 раз із 5, але Llama працювала значно швидше. Тому для простих запитів доцільніше використовувати інструктивну модель, а для складніших задач перевагу може мати міркувальна модель.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Для дослідження впливу system-повідомлення однаковий запит варіанта 10 було виконано моделями llama3.2:3b і qwen3:4b із двома різними системними інструкціями. Спочатку використовувалось повідомлення «Ти лаконічний технічний асистент», а потім – «Відповідай лише українською, максимум двома реченнями».

Результати впливу system-повідомлення на форму відповіді наведено в таблиці 3.

Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	≈1514 символів	переважно українська, з домішками	≈11	частково
llama3.2:3b	українською, 2 речення	313	українська, з помилками	2	так, за форматом
qwen3:4b	лаконічний асистент	283	українська	2	так
qwen3:4b	українською, 2 речення	168	українська	1	так

Як видно з таблиці 3, конкретна вимога щодо мови та максимальної кількості речень виконувалась моделями краще, ніж загальна вказівка бути лаконічним асистентом. Витрати токенів для кожної моделі та кожного system-повідомлення наведено в таблиці 4.

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	59	457	516
llama3.2:3b	українською, 2 речення	63	92	155
qwen3:4b	лаконічний асистент	65	1688	1753
qwen3:4b	українською, 2 речення	74	845	919

З таблиці 4 видно, що після конкретизації обмеження довжини кількість `completion_tokens` зменшилась у обох моделей. Водночас у `qwen3:4b` кількість вихідних токенів залишилась значно більшою, ніж у `Llama3.2:3b`.

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Обидві моделі виконали обмеження «максимум двома реченнями». `Llama3.2:3b` сформуvala два речення, а `qwen3:4b` – одне, тому обидві відповіді відповідали поставленій вимозі.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

При прямій вказівці відповідати лише українською обидві моделі переважно дотримались вимоги. У відповіді `Llama` були граматичні помилки, а `Qwen` використала технічний термін `callback-функцій`, але основна мова залишилась українською.

3. Як змінився `prompt_tokens` при зміні `system-повідомлення` і чому?

У `Llama` `prompt_tokens` збільшився з 59 до 63, а у `Qwen` – з 65 до 74. Це сталося тому, що друге `system-повідомлення` довше і містить більше інструкцій.

4. Чому в `qwen3:4b` значення `completion_tokens` значно більше, ніж у `Llama3.2:3b`, навіть при однаковій довжині видимої відповіді?

`qwen3:4b` є міркувальною моделлю і витрачає частину вихідних токенів на внутрішній процес міркування, який не повністю видно користувачеві. Тому `completion_tokens` може бути великим навіть при короткій кінцевій відповіді.

Відповіді на питання (крок 7)

1. Я порівнював інструктивну модель (`Llama3.2:3b`) та міркувальну (`qwen3:4b`) з різними `system-повідомленнями`. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Різні моделі мають різну архітектуру, спосіб навчання та здатність виконувати інструкції. У моєму експерименті конкретне обмеження «лише українською, максимум двома реченнями» виконувалося краще, ніж загальна вимога бути лаконічним.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

system-повідомлення задає загальні правила поведінки моделі: мову, стиль, формат та інші обмеження, тоді як user-повідомлення містить безпосереднє завдання користувача. Зазвичай system-інструкція має вищий пріоритет, тому модель повинна виконувати user-запит у межах правил, заданих у system.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Міркувальна модель qwen3:4b витрачає частину вихідних токенів на внутрішнє міркування перед формуванням кінцевої відповіді. Тому у моєму експерименті коротка відповідь Qwen мала 845–1688 completion tokens, тоді як у Llama було лише 92–457.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

У нормальному випадку модель повинна дотримуватися інструкції з system, оскільки вона має вищий пріоритет, а суперечливу частину user-повідомлення — ігнорувати. Проте малі моделі можуть виконувати таку ієрархію неідеально й іноді частково слідувати нижчій за пріоритетом інструкції.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Малі моделі мають менше параметрів і, як правило, слабше утримують одночасно зміст запиту та додаткові обмеження щодо мови чи формату. Через це вони частіше змішують мови або допускають дивні формулювання, що ми побачили і в відповідях Llama3.2:3b.

Висновок за завданням 2

System-повідомлення суттєво впливає на стиль, мову та довжину відповіді моделі, але не гарантує ідеального виконання інструкції. У моєму експерименті конкретне обмеження «лише українською, максимум двома реченнями» спрацювало краще, ніж загальна вимога бути лаконічним, тому для стабільного формату інструкції варто формулювати максимально конкретно.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Для порівняння локальної та хмарної моделей було виконано однаковий запит варіанта 10 до llama3.2:3b та gemini-3.6-flash через єдиний OpenAI-сумісний інтерфейс. Відмінності між провайдерами були винесені у файл providers.py, тому основна логіка виклику моделі залишалась однаковою.

Результати порівняння локальної та хмарної моделей наведено в таблиці 5.

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.6-flash
Час відповіді, с	57,37	9,51
completion_tokens	200	291
Довжина відповіді, символів	664	868
Мова відповіді	переважно українська, є мовні помилки та сторонній символ 順	українська
Фактичних помилок у відповіді	є	суттєвих не виявлено

Як видно з таблиці 5, хмарна модель відповіла значно швидше та сформувала більш природну українськомовну відповідь. Локальна модель допустила як мовні, так і змістові неточності.

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

У основному коді виклику моделі нічого змінювати не довелося: функція `create()` використовується однаково для обох провайдерів. Відмінності винесені у словник `PROVIDERS`, де для хмарної моделі задаються інші `base_url`, API-ключ і назва моделі.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь `gemini-3.6-flash` була повнішою і точнішою: вона правильно пояснила синхронне виконання як послідовне з очікуванням завершення попередньої операції, а асинхронне – як неблокуюче. `Llama3.2:3b` некоректно пов'язала асинхронність з виконанням у окремих потоках і допустила плутанин формулювання.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Так. У локальній відповіді були як змістові неточності, так і мовні проблеми: неприродні конструкції, помилки у словах та навіть сторонній символ 順.

4. Хмарна модель відповіла швидше за локальну – за рахунок чого, якщо запит летить через інтернет?

Хмарна модель працює на значно потужнішому серверному обладнанні, оптимізованому для нейромереж, тому час обчислення може бути набагато меншим за локальний. У нашому випадку навіть з мережевою затримкою `Gemini` відповіла за 9,51 с, тоді як локальна `Llama` – за 57,37 с.

Відповіді на питання (крок 6)

1. Чому формат запитів `OpenAI` став стандартом, який підтримують інші провайдери, зокрема локальний сервер `Ollama`?

Формат `OpenAI API` став поширеним через просту та зрозумілу структуру запитів, яку почали підтримувати інші сервіси. Завдяки цьому один і той самий

клієнтський код можна використовувати для різних моделей, змінюючи лише адресу сервера, ключ і назву моделі.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Більша кількість параметрів зазвичай дає моделі більше можливостей для роботи зі складними запитами, але розмір не є єдиним фактором якості. Важливими також є спосіб навчання, якість даних, архітектура моделі та налаштування генерації.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Хмарні моделі працюють на потужному серверному обладнанні, яке спеціально оптимізоване для нейромереж. Тому навіть із передаванням запиту через інтернет загальний час може бути меншим, ніж обчислення на звичайному домашньому комп'ютері.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

При роботі з хмарним API текст запиту передається сторонньому провайдеру, тому не можна безконтрольно надсилати конфіденційні дані. У комерційних проєктах застосовують мінімізацію даних, анонімізацію, контроль доступу та перевіряють політику зберігання інформації у провайдера.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit — це обмеження на кількість запитів або токенів, які можна використати за певний проміжок часу. Розробники зменшують кількість зайвих запитів, використовують кешування, повторні спроби із затримкою або розподіляють навантаження між доступними сервісами.

Висновок за завданням 3

Локальну модель я б обрала у випадках, коли важлива приватність даних, робота без інтернету або відсутність витрат на API. Хмарну модель доцільніше використовувати тоді, коли потрібні вища якість відповіді, краща українська мова та швидше виконання складніших запитів, якщо передавання даних зовнішньому сервісу є допустимим.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Для локальної llama3.2:3b та хмарної gemini-3.6-flash було виміряно час до першого токена (TTFT), загальний час відповіді та швидкість генерації в символах за секунду. Для кожного типу промпту виконувалось три повтори, після чого використовувалась медіана отриманих значень. Перед серією вимірювань виконувався прогрівальний запит.

Результати вимірювання швидкості наведено в таблиці 6.

Таблиця 6 – Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	3,564	11,857	11,8
локальна	llama3.2:3b	середній	3,488	20,601	15,8
локальна	llama3.2:3b	довгий	3,529	156,600	15,7
хмарна	gemini-3.6-flash	короткий	1,847	1,984	19,6
хмарна	gemini-3.6-flash	середній	5,776	6,552	82,6
хмарна	gemini-3.6-flash	довгий	7,841	14,749	241,9

Як видно з таблиці 6, співвідношення між моделями залежить від показника. На короткому промпті хмарна модель мала менший TTFT, однак на середньому й довгому промптах перший токен швидше з'являвся у локальній моделі. Водночас за швидкістю подальшої генерації, особливо для довгого промпту, gemini-3.6-flash значно випередила локальну модель.

Для локальної моделі також було окремо досліджено холодний і теплий старт. Результати наведено в таблиці 7.

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	18,928 с	4,192
Всього, с	20,851 с	14,836

Як видно з таблиці 7, після прогріву TTFT локальної моделі зменшився з 18,928 с до 4,192 с, що свідчить про значний вплив завантаження моделі в пам'ять на перший запуск.

Результати швидкості генерації для різних типів промтів також візуалізовано на рисунку 1.

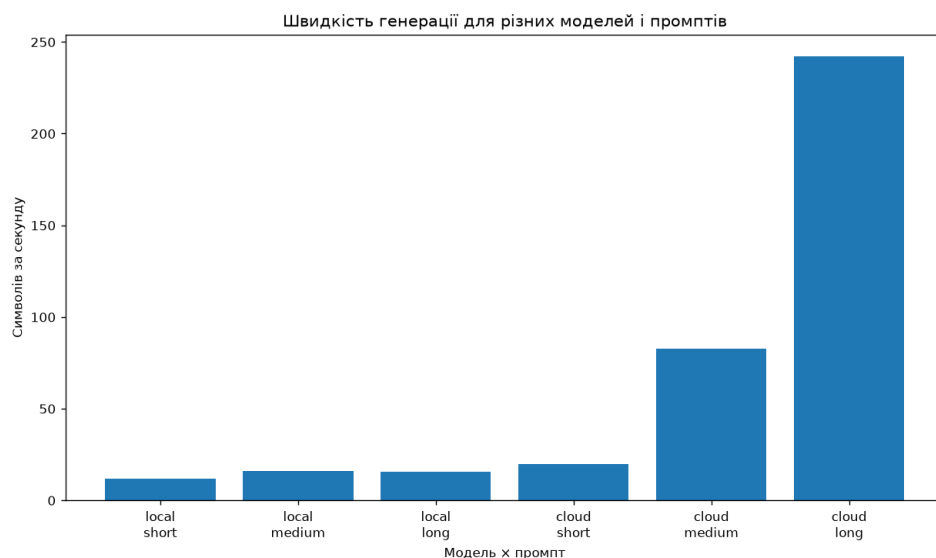


Рисунок 1 – Швидкість генерації локальної та хмарної моделей для промтів різної довжини

Відповіді на питання (крок 6)

1. У якої моделі кращий TTFT і чому саме в неї?

Однозначної переваги немає. Для короткого промпту кращий TTFT мала gemini-3.6-flash – 1,847 с проти 3,564 с, однак для середнього й довгого промптів менший TTFT показала llama3.2:3b: 3,488 і 3,529 с проти 5,776 і 7,841 с відповідно.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Ні. На середньому та довгому промптах локальна модель мала менший TTFT, але хмарна модель генерувала текст значно швидше. Наприклад, на довгому промпті швидкість становила 241,9 симв./с у Gemini проти 15,7 симв./с у Llama.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

На короткому промпті різниця в загальному часі була помітною, але на довгому стала набагато більшою. Локальна модель витратила 156,6 с, тоді як хмарна – лише 14,749 с.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Окремі запуски можуть відрізнятися через кешування, стан моделі, навантаження системи та мережі, тому одного виміру недостатньо. Використання трьох повторів і медіани дає більш стабільний результат і зменшує вплив випадкових затримок.

5. Чому холодний старт повільніший за теплий?

Під час холодного старту модель спочатку завантажується в оперативну пам'ять і готується до роботи, тому TTFT більший. У нашому тесті він зменшився з 18,928 с до 4,192 с після прогріву.

Відповіді на питання (крок 7)

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Під час TTFT система приймає запит, обробляє вхідні токени, виконує початкові обчислення і готує перший вихідний токен. Його вимірюють окремо, бо він показує, скільки користувач чекає до появи першого тексту, а не швидкість подальшої генерації.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Хмарний запит має додаткову мережеву затримку, тому перший токен іноді може з'являтися пізніше. Але після початку генерації потужне серверне обладнання часто видає токени значно швидше за звичайний локальний комп'ютер.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт — це запуск, коли модель ще не завантажена в пам'ять і системі треба підготувати її до роботи. Наступні запити швидші, бо модель уже знаходиться в пам'яті та готова до генерації.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Один запуск може випадково бути повільнішим або швидшим через фонові процеси, кеш, мережу чи навантаження сервера. Медіана кількох вимірів менше залежить від таких випадкових відхилень.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Вхідний текст модель може обробляти великими блоками паралельно, а вихідні токени генеруються послідовно один за одним. Тому створення довгої відповіді зазвичай займає більше часу, ніж аналіз вхідного тексту подібної довжини.

Висновок за завданням 4

Для інтерактивного чат-бота важливішим є TTFT, тому що користувач одразу відчуває затримку до появи першої відповіді. Для фонові обробки великих обсягів

тексту важливішою є швидкість генерації, оскільки саме вона визначає загальний час виконання довгих завдань.

Завдання 5. Оцінювання якості моделей за рубрикою

Для порівняння якості моделей було використано шість типів завдань: перевірка фактичних знань, дотримання формату JSON, стиснення наданого тексту, генерація коду, багатокрокове логічне міркування та якість української мови. Кожне завдання виконувалось двічі для llama3.2:3b, qwen3:4b та хмарної gemini-3.6-flash. Для завдання на фактичні знання правильною відповіддю є 2007 рік. Згідно з офіційним джерелом Apple Newsroom, перший iPhone надійшов у продаж у США 29 червня 2007 року [1]. JSON-відповіді у завданні на дотримання формату були додатково перевірені за допомогою функції json.loads(). Перевірка виконувалась окремо для обох прогонів кожної моделі.

Результати оцінювання моделей за встановленою рубрикою наведено в таблиці 8.

Таблиця 8 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1.Факт	2	2	2
2.Формат	2	2	2
3.Стиснення	0	2	2
4.Код	2	0	1
5.Логіка	0	2	2
6.Українська	0	0	1
Разом /12	6/12	8/12	10/12

Як видно з таблиці 8, найвищий сумарний результат у проведеному експерименті отримала gemini-3.6-flash – 10 балів із 12. qwen3:4b набрала 8 балів,

a llama3.2:3b – 6 балів. Для оцінювання стабільності результатів було порівняно бали двох незалежних прогонів. Результати наведено в таблиці 9.

Таблиця 9 – Розбіжність оцінок між двома прогонами (так \ ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1.Факт	ні	ні	ні
2.Формат	ні	ні	ні
3.Стиснення	ні	ні	ні
4.Код	ні	ні	ні
5.Логіка	ні	ні	ні
6.Українська	ні	ні	ні

Як видно з таблиці 9, у межах проведених двох прогонів оцінки за рубрикою не відрізнялись для жодної з моделей, хоча сам текст відповідей у деяких випадках був різним.

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів набрала gemini-3.6-flash – 10 із 12. Основна різниця між моделями виникла у завданнях на стиснення тексту, генерацію коду, багатокрокову логіку та якість української мови.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

У нашому експерименті оцінки двох прогонів не розійшлися ні в одному завданні для жодної моделі. Це означає, що за цією рубрикою результати виявилися стабільними, хоча самі формулювання відповідей могли відрізнятися.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так. Усі три моделі виконали цю вимогу: їхні JSON-відповіді парсилися через `json.loads()` без жодних правок.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Так. У llama3.2:3b були переходи на інші мови, сторонні символи та кострубаті конструкції, а qwen3:4b також змішувала українську з англійською та мала неприродні формулювання. gemini-3.6-flash писала значно природнішою українською, хоча мала окремі неточності в термінології.

Крок 7. Приклад провалу

Повний текст запиту: сервер обробляє 1200 запитів/хв, 15% дають помилку, з них 40% повторюються успішно. Скільки запитів втрачено за годину? Покажи хід розв'язання.

Повний текст відповіді моделі llama3.2:3b:

Дякуюм за запит!

Зрозумію, що нам потрібно знайти число запитів, які втрачаються за годину.

Початківий, ми маємо наступність істот:

1. Обробка запитів на хвилину: 1200 запитів/хв.
2. Суспільний рівень помилок: 15%.
3. Суспільний рівень повторювання успішно: 40%.

Вихід з цієї проблеми можна здійснити за допомогою наступного ходу:

1. Кількість помильних запитів на хвилину: $15\% * 1200 \text{ запитів/хв} = 0,15 * 1200 = 180 \text{ запитів/хв}$.

2. Кількість повторно успішно оброблених запитів на хвилину: $40\% * 1200 \text{ запитів/хв} = 0,4 * 1200 = 480 \text{ запитів/хв}$.

3. Кількість втрачених запитів на хвилину: $180 \text{ запитів/хв} - 480 \text{ запитів/хв} = -300 \text{ запитів/хв}$.

4. Кількість втрачених запитів на годину: $-300 \text{ запитів/хв} * 3600 \text{ хв в годі} = -1080000 \text{ запитів в годину}$.

Теоретически, за годину втрачається 1080000 запитів.

Правильна відповідь: 6480 запитів за годину.

Правильний хід: $1200 \times 15\% = 180$ помилкових запитів за хвилину. Із них 40% повторюються успішно, тобто втрачаються 60%: $180 \times 0,60 = 108$ запитів за хвилину. За годину: $108 \times 60 = 6480$.

Модель загубилася в багатокроковій арифметиці та неправильно застосувала 40% до всіх 1200 запитів замість того, щоб застосувати їх лише до 180 помилкових. Після цього помилка накопичилася в наступних кроках, тому модель отримала негативну кількість втрачених запитів і неправильну кінцеву відповідь. Це показує, що правдоподібно оформлений хід розв'язання не гарантує правильності обчислень.

Відповіді на питання (крок 8)

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Крім ручної рубрики, використовують автоматичні тести, набори еталонних відповідей, спеціальні бенчмарки та порівняння відповідей кількох моделей. Також застосовують оцінювання людьми або іншою моделлю за наперед визначеними критеріями.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Модель навчалася на великій кількості прикладів, де код і JSON часто оформлені в markdown-блоках. Через це звичний шаблон відповіді інколи виявляється сильнішим за окрему інструкцію користувача.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

У навчальних даних зазвичай значно більше англomовних текстів, ніж україномовних, а мала модель має меншу здатність узагальнювати рідше представлені мовні конструкції. Через це частіше виникають русизми, кальки, неприродні фрази та змішування мов.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Генерація мовної моделі має випадковий компонент, особливо коли temperature більша за нуль. Тому при однаковому запиті модель може обрати різні наступні токени й побудувати інший хід відповіді.

5. Що таке галюцинація мовної моделі і чому модель радше вигадас факт, ніж визнає незнання?

Галюцинація – це правдоподібна, але неправильна або вигадана відповідь. Модель генерує найбільш імовірне продовження тексту і не має вбудованого механізму перевірки істинності кожного факту, тому іноді впевнено формулює те, чого насправді не знає.

Висновок за завданням 5

Оцінювання за рубрикою виявилось достатньо надійним, оскільки дозволило порівнювати моделі за однаковими критеріями, а не за суб'єктивним враженням. Найпоказовішими були завдання на логіку, формат JSON та українську мову, оскільки саме в них добре проявилися відмінності між моделями. Для роботи з українськомовним текстом я б обрала gemini-3.6-flash, оскільки вона показала найкращу якість мови та найвищий сумарний результат – 10 із 12 балів.

Завдання 6. Керування випадковістю: temperature і top_p

Експерименти з параметрами temperature і top_p виконувались на локальній моделі llama3.2:3b. Для творчого завдання використовувалось генерування назви застосунку для пішохідних екскурсій, а для фактичного – питання про рік відкриття будівлі Одеського оперного театру. Для фактичного завдання правильною відповіддю є 1887 рік. Згідно з офіційною історією Одеського національного академічного театру опери та балету, будівництво нової будівлі завершилося у вересні 1887 року, а 1 жовтня 1887 року глядачі вперше заповнили залу [2].

Результати впливу temperature та top_p на різноманітність відповідей наведено в таблиці 10.

Таблиця 10 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0
temperature	0.7	5	-
temperature	1.5	5	0
top_p	0.1	1	-
top_p	0.5	5	-
top_p	1.0	5	-

Як видно з таблиці 10, при temperature = 0,0 усі п'ять творчих відповідей були однаковими, тоді як при temperature = 0,7 і 1,5 було отримано по п'ять різних відповідей. Подібна закономірність спостерігалась і для top_p: при top_p = 0,1 усі відповіді збігались, а при 0,5 та 1,0 усі п'ять відповідей були різними. На фактичному завданні при temperature = 0,0 модель п'ять разів дала однакову, але неправильну відповідь 1856. При temperature = 1,5 було отримано різні роки – 1876, 1849, 1856, 1907 і 1846, однак правильної відповіді 1887 не було жодного разу. Отже, низька температура підвищила стабільність відповіді, але не забезпечила її фактичну правильність.

Для окремої перевірки відтворюваності фактичний запит було виконано десять разів при temperature = 0,0. Результати наведено в таблиці 11.

Таблиця 11 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

У всіх десяти повторних запусках модель дала одну й ту саму відповідь 1856. Таким чином, у проведеній серії відповідь була повністю відтворюваною, але фактично неправильною.

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Я б поставила низьку температуру, наприклад 0.0 або близьке до нуля значення. Для JSON важлива передбачуваність і стабільність формату, а висока температура збільшує випадковість відповіді.

2. Яке — генератору варіантів заголовків?

Для генератора заголовків доцільно використати вищу температуру, наприклад 0.7–1.0. У моєму експерименті при 0.7 і 1.5 було по 5 унікальних відповідей із 5, тоді як при 0.0 модель повторювала одну й ту саму відповідь.

3. Чим вплив `top_r` відрізняється від впливу `temperature` за характером, а не за силою?

`temperature` змінює сам розподіл ймовірностей, роблячи вибір більш або менш випадковим. `top_r` не змінює співвідношення ймовірностей, а обмежує набір токенів, з яких модель може вибирати наступний.

4. Чи означає `temperature=0` повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Повної математичної гарантії це не дає, але в моєму експерименті всі 10 запусків при `temperature=0.0` дали одну й ту саму відповідь 1856. Отже, у цій серії результат був повністю відтворюваним.

5. Наведіть найбезглуздішу відповідь, яку модель видала при `temperature=1.5`, і поясніть, чому вона взагалі могла з'явитися.

Однією з найбезглуздіших відповідей була назва «Теребре», яка не має зрозумілого змісту в контексті застосунку пішохідних екскурсій. Вона могла

з'явитися через високу температуру, яка збільшує ймовірність вибору малоїмовірних токенів і незвичних комбінацій слів.

Відповіді на питання (крок 8)

1. Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена?

`temperature` змінює “різкість” розподілу ймовірностей перед вибором наступного токена. Низьке значення робить найбільш імовірні токени ще домінантнішими, а високе — вирівнює розподіл і збільшує шанс вибору менш імовірних варіантів.

2. Чим `top_p` відрізняється від `temperature` — обидва впливають на різноманітність, але за яким механізмом?

`temperature` змінює самі відносні ймовірності токенів, а `top_p` відсікає малоїмовірний “хвіст” і залишає лише найімовірніші токени, сумарна ймовірність яких досягає заданого порогу. Тому механізми різні, хоча обидва впливають на різноманітність.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

На практиці можна змінювати обидва, але для зрозумілого налаштування краще спочатку змінювати один параметр, залишаючи інший фіксованим. Так легше зрозуміти, який саме параметр викликав зміну поведінки моделі.

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Навіть при `temperature=0` на результат можуть впливати особливості обчислень з плаваючою комою, паралелізація та реалізація самого inference. У моєму експерименті всі 10 запусків були однаковими, але це не є універсальною гарантією.

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для структурованих даних зазвичай використовують низькі значення, близькі до 0.0, щоб зменшити випадковість і зробити формат стабільнішим. Для

творчих задач частіше використовують середні або високі значення, наприклад 0.7–1.0, щоб отримувати більше різних варіантів.

Висновок за завданням 6

Для наступних лабораторних, де модель має повертати структуровані дані, я б використовувала низьку temperature, близьку до 0.0, і помірний top_p, щоб зменшити випадковість і підвищити стабільність формату. Мій експеримент показав, що при temperature=0.0 модель давала однакову відповідь у всіх 10 запусках, тоді як при temperature=1.5 відповіді ставали значно різноманітнішими й інколи беззмистовними. Налаштування за замовчуванням може бути непридатним, тому що воно не враховує конкретну задачу: для JSON потрібна передбачуваність, а для творчої генерації – різноманітність.

Підсумкове порівняння моделей

Узагальнені результати всіх проведених експериментів наведено в таблиці 12.

Таблиця 12 – Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	У benchmark: приблизно 11,8–15,8 симв./с	Повний benchmark окремо не проводився; у ранньому фактичному тесті $\approx$ 4,91 ток./с, модель працювала значно повільніше	У benchmark: приблизно 19,6–241,9 симв./с, найвища швидкість на довгому запиті

TTFT	Приблизно 3,49–3,56 с у benchmark	Окремо не вимірювався	1,85 с для короткого, 5,78 с для середнього, 7,84 с для довгого промпту
Якість, балів /12	6/12	8/12	10/12
Стабільність формату (завдання 2)	Висока: обидва JSON парсилися через json.loads() без правок	Висока: обидва JSON парсилися без правок	Висока: обидва JSON парсилися без правок
Потрібні ресурси	Працює на локальному ПК; потребує оперативної пам'яті та обчислювальних ресурсів	Потребує більше локальних ресурсів і в моїх тестах працювала значно повільніше	Основні обчислення виконуються на сервері провайдера; локальні вимоги мінімальні
Вартість	Немає оплати за кожен API-запит, але використовуються ресурси та електроенергія власного ПК	Так само: немає оплати за локальний запит, але потрібні більші ресурси ПК	Залежить від тарифу хмарного API; може існувати оплата за використання
Куди йдуть ваші дані	Залишаються на локальному комп'ютері	Залишаються на локальному комп'ютері	Запит передається зовнішньому хмарному провайдеру
Працює без інтернету	Так	Так	Ні

Коли обирати саме її	Коли важлива приватність, автономність, робота без інтернету й задача не потребує максимальної якості	Коли потрібна локальна модель з кращою якістю міркування, ніж у малої моделі, і допустимі більші витрати часу та ресурсів	Коли пріоритетом є якість, швидка генерація великих відповідей і краща робота з українською мовою, а доступ до інтернету та передача запитів зовнішньому сервісу допустимі
----------------------	---	---	--

У таблиці 12 важливо залишити примітку, що для Qwen 4,91 ток./с не можна напряду порівнювати з симв./с, бо це різні одиниці. Тому формулювання «окремий benchmark у симв./с не проводився» тут правильне.

Загальні висновки

1) Локальна та хмарна моделі можуть використовуватись через єдиний OpenAI-сумісний програмний інтерфейс, тому перехід між провайдерами не потребує зміни основної логіки програми. Це дозволяє будувати системи з можливістю перемикавання між локальним і хмарним виконанням.

2) Продуктивність мовної моделі необхідно оцінювати окремо за TTFT та швидкістю подальшої генерації. У проведеному експерименті Llama3.2:3b мала менший TTFT на середньому та довгому промптах, однак gemini-3.6-flash значно перевершувала її за швидкістю генерації довгих відповідей. Отже, одного показника недостатньо для висновку про те, яка модель є «швидшою».

3) Міркувальний режим є доцільним насамперед для задач із багатокроковими логічними або математичними обчисленнями. У простому фактичному запиті qwen3:4b витратила значно більше токенів і часу, тоді як у задачі на міркування правильно відповіла у всіх п'яти запусках.

4) Текстова інструкція не гарантує точного дотримання заданого формату. У завданні на генерацію коду qwen3:4b додавала значний обсяг зайвого тексту, а

gemini-3.6-flash використовувала markdown-оформлення. Тому у випадках, коли відповідь має використовуватись програмою, формат необхідно перевіряти автоматично.

5) Мовні моделі можуть давати різні результати при повторному виконанні однакового запиту, тому одиничного запуску недостатньо для надійного вимірювання. Для продуктивності в роботі використовувалось кілька повторів із медіаною, а для оцінювання якості кожне завдання виконувалось двічі.

6) Низьке значення temperature підвищує стабільність відповіді, але не гарантує її правильності. У проведеному експерименті при temperature = 0,0 модель десять разів поспіль дала однакову неправильну відповідь 1856, тоді як при temperature = 1,5 відповіді стали різноманітнішими, але також залишались неправильними.

7) Оцінювання за рубрикою показало різний профіль моделей: llama3.2:3b отримала 6/12, qwen3:4b – 8/12, а gemini-3.6-flash – 10/12. Найпомітніші відмінності проявились у завданнях на багатокрокову логіку, генерацію коду, роботу з наданим текстом та якість української мови.

8) Вибір між локальною та хмарною моделлю залежить від вимог конкретної задачі. Локальну модель доцільно використовувати, якщо пріоритетними є приватність, автономність і робота без інтернету. Хмарну модель варто обирати тоді, коли важливішими є якість відповіді, швидкість генерації та краща робота зі складними або україномовними запитамі, а передавання даних зовнішньому провайдеру є допустимим.

Список використаних джерел

1. Apple Newsroom. iPhone Premieres This Friday Night at Apple Retail Stores. June 28, 2007. <https://www.apple.com/newsroom/2007/06/28iPhone-Premieres-This-Friday-Night-at-Apple-Retail-Stores/>

2. Одеський національний академічний театр опери та балету. Історія театру. <https://operahouse.od.ua/about/history/>